BotaniX AI: Flower Species Classification using Deep Learning and Transfer Learning

Deep Learning Semester Project Report

Submitted by: Muhammad Faizan **Roll Number:** SU92-BSCSM-F23-169

Department: Department of Computer Science

University: The Superior University

Course: Deep Learning **Instructor:** Dr Munir Ahmad

Abstract

This project builds a deep learning system that identifies the species of a flower from a single photograph. It uses the Oxford 102 Flowers dataset, which contains 8,189 images across 102 flower species. A MobileNetV2 network pretrained on ImageNet was fine-tuned through transfer learning for ten epochs on Google Colab (T4 GPU) in about 8.4 minutes. On 1,229 test images that the model never saw during training, it reached an accuracy of 97.72% and a macro F1-score of 0.9725. The trained model was exported to ONNX format (9.4 MB) and deployed in a web application that runs entirely in the browser using ONNX Runtime Web, so users can upload a photo and receive the top three predicted species with confidence scores. The source code, trained weights and the web application are published in a public GitHub repository and on Vercel.

1. Introduction and Problem Statement

Recognising flowers is difficult even for people. Many species look alike in colour, petal shape and size, and the same species can appear very different depending on lighting, background, angle and growth stage. Botanists, gardeners, students and ordinary nature lovers often need a fast way to name a flower they have photographed.

The problem addressed in this project is **multi-class image classification**: given an RGB photograph of a flower, predict which of 102 species it belongs to. Training such a model from scratch needs a large amount of labelled data and computation. Transfer learning avoids this by starting from a network that has already learned general visual features (edges, textures, shapes) on a very large dataset and adapting it to the new task.

2. Objectives

- Select a suitable secondary dataset and prepare it for deep learning.
- Train and evaluate a convolutional neural network for 102-class flower classification using transfer learning.
- Save the trained model weights in a reusable format.
- Deploy a web interface where a user can upload a flower photo and see the predicted species and confidence.

- Publish the code, weights and documentation in a public GitHub repository.

3. Dataset Description

The project uses the **Oxford 102 Flowers** dataset created by the Visual Geometry Group at the University of Oxford (Nilsback and Zisserman, 2008). It is a secondary dataset that is freely available for research and is loaded directly through torchvision.

Property	Value
Total images	8,189
Number of classes	102 flower species
Images per class	between 40 and 258
Image type	colour (RGB) photographs with varied size, lighting and background
Official split	1,020 train, 1,020 validation, 6,149 test

The official training split contains only 1,020 images, which is small for fine-tuning a deep network. For this reason the three official splits were merged (1,020 + 1,020 + 6,149 = 8,189 images) and divided again at random into 70% training, 15% validation and 15% test data, using a fixed random seed (42) so that the split is reproducible.

Split	Images	Purpose
Training	5,732	Learning the model weights
Validation	1,228	Choosing the best epoch
Test	1,229	Final, unbiased evaluation

4. Methodology

4.1 Preprocessing

All images were converted to tensors and normalised with the ImageNet channel means (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225), because the pretrained network was trained with this normalisation. Validation and test images were resized so that the shorter side is 256 pixels and then centre-cropped to 224 x 224 pixels.

4.2 Data Augmentation

To reduce overfitting, the training images (and only the training images) were randomly transformed every time they were loaded:

- Random resized crop to 224 x 224 (scale between 0.7 and 1.0)
- Random horizontal flip
- Random rotation of up to 15 degrees
- Colour jitter (brightness, contrast and saturation of 0.2)

Validation and test images were not augmented, so that the evaluation is consistent.

4.3 Model Architecture

The model is **MobileNetV2** (Sandler et al., 2018), a lightweight convolutional network designed for mobile and embedded devices. Its main ideas are:

- **Depthwise separable convolutions**, which split a standard convolution into a per-channel spatial filter and a 1 x 1 channel-mixing convolution. This greatly reduces the number of parameters and computations.
- **Inverted residual blocks with linear bottlenecks**, which expand the number of channels, filter them, and compress them again, with a shortcut connection between the narrow layers.

The network was initialised with weights pretrained on ImageNet. Its original 1,000-way classification layer was replaced by a new linear layer that maps the 1,280 features to 102 outputs, one per flower species. All layers were fine-tuned, not only the new layer. The resulting model has roughly 2.4 million parameters, which is why its exported size is only 9.4 MB.

Why MobileNetV2? Larger networks such as ResNet50 give good accuracy but have much bigger weight files (about 98 MB), which is close to the GitHub file-size limit and heavy to download in a web page. MobileNetV2 is small, fast to train, fast to run in a browser, and still very accurate on this dataset.

4.4 Training Setup

Setting	Value
Framework	PyTorch and torchvision
Hardware	Google Colab, NVIDIA T4 GPU
Loss function	Cross-entropy
Optimiser	AdamW
Initial learning rate	3e-4
Learning-rate schedule	Cosine annealing over 10 epochs
Batch size	32
Epochs	10
Model selection	Weights with the highest validation accuracy were kept
Training time	about 8.4 minutes

After each epoch the model was evaluated on the validation set, and the weights of the best epoch were saved. The test set was used only once, at the end.

5. Results

5.1 Training Progress

Epoch	Train loss	Train accuracy	Validation loss	Validation accuracy
1	1.764	0.634	0.488	0.902
2	0.357	0.928	0.283	0.931
3	0.188	0.965	0.204	0.952
4	0.094	0.983	0.159	0.959
5	0.061	0.990	0.140	0.968
6	0.034	0.995	0.114	0.974
7	0.025	0.997	0.114	0.976
8	0.016	0.999	0.105	0.976
9	0.014	0.999	0.106	0.976
10	0.013	0.999	0.107	0.976

The training loss drops quickly in the first three epochs, and the validation accuracy is already 90.2% after the very first epoch, which shows how useful the pretrained features are. Both curves flatten after about epoch 7. The validation accuracy stays near 97.6% while the training accuracy approaches 99.9%. This small gap indicates mild overfitting, which is normal for fine-tuning on a dataset of this size and did not harm the validation or test results.

5.2 Test Set Performance

Metric	Value
Test images	1,229
Test accuracy	97.72%
Macro F1-score	0.9725
Misclassified images	28

The macro F1-score is the unweighted average of the F1-score of each of the 102 classes. It is close to the accuracy, which shows that the model performs well across all species and not only on the most frequent ones.

5.3 Error Analysis

Of the 1,229 test images, 28 were classified incorrectly. The most frequent confusions were:

True species	Predicted species	Count
Petunia	Tree mallow	2
Rose	Camellia	1
Garden phlox	Petunia	1
Prince of Wales feathers	Carnation	1
Tree poppy	Windflower	1
Sword lily	Canna lily	1
Sword lily	Cyclamen	1
Japanese anemone	Tree poppy	1

Many of these mistakes are between species that are visually similar, such as sword lily and canna lily, or rose and camellia, where petal arrangement and colour overlap. The errors are spread over many different classes rather than concentrated in one, which agrees with the high macro F1-score.

6. Web Application and Deployment

6.1 Model Export

The best PyTorch weights were exported to the **ONNX** (Open Neural Network Exchange) format as a single 9.4 MB file. The ONNX output was checked against the PyTorch output on test images to confirm that both give the same predictions.

6.2 In-Browser Inference

The web application is a static website (HTML, CSS and JavaScript) that loads the ONNX model with **ONNX Runtime Web**. When a user selects a photo, JavaScript performs the same preprocessing as in training (resize, centre crop to 224 x 224, ImageNet normalisation), runs the model, applies softmax and displays the top three species with their confidence. If the highest confidence is below 50%, the page warns that the photo may be unclear or not one of the 102 species.

Because inference happens inside the browser, no server or API is needed and the uploaded photo never leaves the user's device. This also makes deployment simple and free.

6.3 Deployment

The site is hosted on **Vercel**. Besides the live demo, it presents the dataset and code details and links to this report, the presentation, the YouTube video and the GitHub repository.

During testing, an uploaded photograph of an oxeye daisy was identified correctly as *oxeye daisy* with 96.4% confidence.

7. Repository and Reproducibility

The public GitHub repository contains:

- the full training notebook with its outputs,
- `src/predict.py` to classify a single image and `src/evaluate.py` to recompute the test accuracy and F1-score,
- the trained weights (`best_model.pth`) and the ONNX model,
- the class name list, training curves and example predictions,
- the web application source and a README with setup instructions and a link to the dataset.

The fixed random seed and the documented hyperparameters allow the data split and the experiment to be reproduced.

8. Limitations

- The model knows only the 102 species in the dataset. For any other flower, or for a non-flower image, it will still output the closest known species, so the confidence warning is only a partial safeguard.
- The dataset photographs are fairly clean. Accuracy may be lower on blurry photos, photos with several flowers, or unusual backgrounds.
- Only one architecture (MobileNetV2) was trained, so no comparison with other models is reported.
- Performance was measured on one random split. Repeating the experiment with other seeds would give a better estimate of the variation.

9. Conclusion and Future Work

This project showed that transfer learning with a small network can classify 102 flower species with high accuracy: 97.72% test accuracy and a macro F1-score of 0.9725, after only about 8.4 minutes of training. The model was exported to ONNX and delivered as a lightweight web application that runs fully in the browser, and all code and weights are openly available.

Possible improvements include comparing MobileNetV2 with EfficientNet and ResNet models, using Grad-CAM to visualise which parts of the flower the model relies on, adding a class for not a flower, detecting several flowers in one image, and adding short descriptions or care tips for each predicted species.

References

1. Nilsback, M.-E. and Zisserman, A. (2008). Automated Flower Classification over a Large Number of Classes. Indian Conference on Computer Vision, Graphics and Image Processing (ICVGIP).
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
3. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. CVPR.
4. Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. International Conference on Learning Representations (ICLR).
5. Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. ICLR.

6. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS.
7. ONNX Runtime Web documentation, https://onnxruntime.ai/
8. Oxford 102 Flowers dataset, https://www.robots.ox.ac.uk/~vgg/data/flowers/102/